dragmapr cheatsheet

Drag map regions, save the state, render it again.

v0.3

What it does

dragmapr is for editorial map composition. Move regions and labels by hand, then save the offsets as reproducible data.

- drag regions
- drag labels
- save JSON state or offset CSVs
- render the same layout later

Quick start

```
library(dragmapr)

my_sf <- prepare_dragmapr_sf(my_sf)

drag_map_prototype(
  my_sf,
  region_col = "region",
  open = TRUE
)
```

Download the offset CSVs, then render:

```
render_dragged_map(
  my_sf,
  region_col = "region",
  region_offsets = "drag_region_offsets.csv",
  label_offsets = "drag_label_offsets.csv",
  file = "map.png"
)
```

State-first workflow

For new apps, carry one `dragmapr_state` through editing and rendering.

```
library(explodemap)
library(dragmapr)

layout <- explode_grouped(
  my_sf,
  region_col = "region",
  plot = FALSE
)

state <- as_dragmapr_state(layout)
d_edit(layout, state = state)
```

```
focus_map(layout, state = state)

render_dragged_map(
  layout$sf_grouped,
  region_col = "region",
  state = state
)
```

Save and restore

```
write_dragmapr_state(
  state,
  "composition.json"
)

state <- read_dragmapr_state(
  "composition.json"
)
```

Shiny widget

```
ui <- fluidPage(
  dragmaprOutput("map")
)

server <- function(input, output, session) {
  output$map <- renderDragmapr({
    d_widget(
      my_sf,
      region_col = "region",
      region_palette = palette
    )
  })

  observeEvent(input$map_state, {
    state <- d_widget_state(
      input$map_state
    )
  })
}
```

Live updates

```
updateDragmapr(
  session,
  "map",
  selected_feature = "North"
)

updateDragmapr(
  session,
  "map",
  region_palette = palette
)
```

Input checklist

- polygon or multipolygon `sf`
- projected CRS
- one column that names regions
- simplify large geometry before browser editing

Compare states

```
diff <- d_state_diff(
  draft,
  canonical,
  tolerance = 1
)

diff$changed_regions
diff$changed_labels

d_state_equal(
  draft,
  canonical,
  compare = "composition"
)
```

> Use `compare = "composition"` when selection or viewport changes should not count as dirty edits.

Labels and notes

```
d_widget(
  my_sf,
  region_col = "region",
  labels = FALSE
)

labels <- make_region_labels(
  my_sf,
  region_col = "region",
  label_col = "name"
)
```

```
notes <- as_drag_annotations(
  notes_df,
  connector = TRUE,
  connector_type = "curve"
)
```

Apps

Spatial Studio	`examples/shiny_spatial_studio.R`
Pipeline Studio	`shiny/pipeline-studio`
Round trip	`examples/full_state_roundtrip.R`